

Actividades Práctico-Experimentales

Nro. 002

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	Estructuras de datos estáticas (arreglos, conjuntos, registros, archivos) y estructuras de datos dinámicas (listas, pilas y colas)
Resultado de aprendizaje de la unidad	R1. Implementa algoritmos simples de búsqueda y explica las diferencias en el orden de complejidad computacional, bajo los principios de solidaridad,
Práctica Nro.	02
Título de la Práctica	Implementación de acceso secuencial y aleatorio a archivos con métricas de I/O
Nombre del Docente	Cristian Ramiro Narváez Guillén
Integrantes del Grupo	Isaac Vire Axel Jiménez Danny Beltrán Lucas Orellana Noelia Bustan
Fecha	jueves 8 de octubre 2026 - viernes 9 de octubre 2026.
Horario	jueves 8 de 08:30 – 09:30 viernes 9 de 11:30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Implementar algoritmos de acceso secuencial para la persistencia masiva de datos en disco mediante el módulo nativo de I/O en JavaScript.

Cuantificar empíricamente la latencia computacional (tiempo de respuesta $T(n)$) y el impacto en la memoria Heap ($S(n)$) al procesar archivos de forma síncrona (bloqueante) versus flujos asíncronos (*Streams*).

Evaluar las decisiones de diseño físico en el acceso a almacenamiento mecánico o de estado sólido bajo los principios de *Green Computing*, analizando cómo la optimización de los accesos I/O reduce el consumo energético del hardware.

3. Materiales y reactivos:

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación técnica oficial de Node.js (módulo fs).
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).
- Recursos Digitales: Sílabo de la asignatura, repositorio base en GitHub.
- Plataforma EVA UNL.

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos (recomendado 4).
- Memoria RAM: Mínimo 4 GB.
- Almacenamiento: Disco HDD o SSD con al menos 1 GB de espacio libre temporal.

5. Procedimiento / Metodología.

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** El equipo satelital del APE 001 ahora necesita leer el millón de coordenadas ($N = 10^6$) que han sido transmitidas y almacenadas en un archivo físico en el servidor. El problema radica en que leer el archivo de una sola vez colapsa la memoria RAM del contenedor en la nube. Se debe diseñar una estrategia para leer los datos con el mínimo impacto temporal y espacial.

- **ABI (Aprendizaje Basado en Investigación):** El estudiante investigará cómo interactúa el *Event Loop* de JavaScript (motor V8 y libuv) con las llamadas del sistema operativo (Syscalls) para el acceso a disco.
- **Modalidad:** Individual y análisis comparativo grupal posterior.

Tarea 1: Generación del Conjunto de Datos Masivo (Mock Data)

Para poder medir el I/O, primero necesitamos un archivo físico considerable.

1. En el directorio de su proyecto, cree un archivo llamado `lab02_generador.js`.
2. Implemente el siguiente código para escribir un archivo CSV con 10^6 registros utilizando escritura en flujo (Stream) para no saturar la memoria durante su creación.

```
const fs = require('fs');
const { performance } = require('perf_hooks');

const FILE_NAME = 'coordenadas_masivas.csv';
const N = 1000000;

console.log(`[Generador] Iniciando escritura de ${N} registros...`);
const start = performance.now();

// Utilizamos un Stream de escritura (Write Stream)
const writableStream = fs.createWriteStream(FILE_NAME);
writableStream.write('id,latitud,longitud\n'); // Cabecera

for (let i = 0; i < N; i++) {
  const lat = (Math.random() * 180 - 90).toFixed(6);
  const lng = (Math.random() * 360 - 180).toFixed(6);
  // Buffer string format
  writableStream.write(`${i},${lat},${lng}\n`);
}

writableStream.end();

writableStream.on('finish', () => {
  const end = performance.now();
  console.log(`[Generador] Archivo creado en ${((end - start) / 1000).toFixed(2)} segundos.`);
  // Calcular tamaño del archivo en disco
  const stats = fs.statSync(FILE_NAME);
  const sizeInMB = stats.size / (1024 * 1024);
```

```
console.log(`[Generador] Peso físico en disco: ${sizeInMB.toFixed(2)} MB`);
});
```

3. Ejecute en terminal: `node lab02_generador.js` y verifique la creación del archivo `coordenadas_masivas.csv`.

Tarea 2: Enfoque 1 - Lectura Síncrona (Bloqueante / $O(n)$ Espacial)

Este enfoque representa la manera más intuitiva, pero menos eficiente, de procesar un archivo físico.

1. Cree un archivo llamado `lab02_lectura_sync.js`.
2. Implemente el código para cargar todo el archivo en la RAM.

```
const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
console.log(`[Lectura Síncrona] Iniciando carga en memoria...`);
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
// I/O Bloqueante: El hilo principal se detiene hasta cargar todo
const data = fs.readFileSync(FILE_NAME, 'utf-8');
// Simulamos procesamiento separando por saltos de línea
const lineas = data.split('\n');
const end = performance.now();
const memoryAfter = process.memoryUsage().heapUsed;
console.log(`[Lectura Síncrona] Total registros leídos: ${lineas.length - 1}`);
console.log(`[Lectura Síncrona] Tiempo de I/O + Parsing: ${((end - start) / 1000).toFixed(2)} segundos.`);
console.log(`[Lectura Síncrona] Consumo Neto de RAM: ${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB`);
```

3. Ejecute `node lab02_lectura_sync.js` y tabule el tiempo y el impacto en memoria.

Tarea 3: Enfoque 2 - Lectura Asíncrona con Streams ($O(1)$ Espacial)

1. Cree el archivo `lab02_lectura_stream.js`.
2. Implemente la abstracción de flujos (Streams) para leer fragmentos (*chunks*) secuenciales,

reduciendo la complejidad espacial teórica de $S(n) = n \cdot \text{bytes}$ a un constante

$S(n) = O(1)$ correspondiente al tamaño del *buffer*.

```
const fs = require('fs');
const { performance } = require('perf_hooks');
```

```

const FILE_NAME = 'coordenadas_masivas.csv';
console.log(`[Lectura por Stream] Iniciando procesamiento...`);
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
let totalRegistros = 0;
// Creamos un flujo de lectura secuencial no bloqueante
const readableStream = fs.createReadStream(FILE_NAME, { encoding: 'utf-8' });
readableStream.on('data', (chunk) => {
  // Procesamos el chunk de datos actual sin almacenarlo completo
  // Contamos cuántos saltos de línea hay en este fragmento
  let lineBreakCount = (chunk.match(/\n/g) || []).length;
  totalRegistros += lineBreakCount;
});
readableStream.on('end', () => {
  const end = performance.now();
  const memoryAfter = process.memoryUsage().heapUsed;
  console.log(`[Lectura por Stream] Total registros procesados: ${totalRegistros}`);
  console.log(`[Lectura por Stream] Tiempo de I/O parcializado: $${(end - start) / 1000}.toFixed(2)} segundos.`);
  console.log(`[Lectura por Stream] Consumo Neto de RAM: $${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB`);
});

```

3. Ejecute `node lab02_lectura_stream.js`. Compare drásticamente la métrica del consumo neto de RAM entre la Tarea 2 y la Tarea 3.

Tarea 4: Análisis Matemático de Rendimiento (ABI)

Basado en los resultados empíricos, los estudiantes investigarán el diseño del bus de datos y el *Thread Pool* interno (libuv) que gestiona estas llamadas.

- Demuestre teóricamente (utilizando notación de orden asintótico) por qué la función bloqueante impone $S(n) = O(n)$ frente al enfoque *Stream* $S(n) = O(1)$.
- Estructure su respuesta relacionándola con el uso energético del servidor (menos RAM implica menor recolección de basura y menor gasto térmico del procesador).

6. Resultados obtenidos por el grupo:

Resultados de la Tarea 1:

```

const fs = require('fs');
const { performance } = require('perf_hooks');

const FILE_NAME = 'coordenadas_masivas.csv';
const N = 1000000;

console.log(`[Generador] Iniciando escritura de ${N} registros...`);
const start = performance.now();

// Stream de escritura
const writableStream = fs.createWriteStream(FILE_NAME);
writableStream.write('id,latitud,longitud\n');

for (let i = 0; i < N; i++) {
  const lat = (Math.random() * 180 - 90).toFixed(6);
  const lng = (Math.random() * 360 - 180).toFixed(6);
  // Buffer string format
  writableStream.write(`${i},${lat},${lng}\n`);
}

writableStream.end();

writableStream.on('finish', () => {
  const end = performance.now();
  console.log(`[Generador] Archivo creado en ${((end - start) / 1000).toFixed(2)} segundos.`);
  // Calcular tamaño del archivo en disco
  const stats = fs.statSync(FILE_NAME);
  const sizeInMB = stats.size / (1024 * 1024);
  console.log(`[Generador] Peso físico en disco: ${sizeInMB.toFixed(2)} MB`);
});

```

```

[Generador] Iniciando escritura de 1000000 registros...
[Generador] Archivo creado en 8.50 segundos.
[Generador] Peso físico en disco: 26.86 MB

```

Resultados de la Tarea 2:

```

const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
console.log(`[Lectura Síncrona] Iniciando carga en memoria...`);
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
// El hilo principal se detiene hasta cargar todo
const data = fs.readFileSync(FILE_NAME, 'utf-8');

const lineas = data.split('\n');
const end = performance.now();
const memoryAfter = process.memoryUsage().heapUsed;
console.log(`[Lectura Síncrona] Total registros leídos: ${lineas.length - 2}`);
console.log(`[Lectura Síncrona] Tiempo de I/O + Parsing: ${((end - start) / 1000).toFixed(2)} segundos.`);
console.log(`[Lectura Síncrona] Consumo Neto de RAM: ${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB`);

```

```

[Lectura Síncrona] Iniciando carga en memoria...
[Lectura Síncrona] Total registros leídos: 1000000
[Lectura Síncrona] Tiempo de I/O + Parsing: 0.12 segundos.
[Lectura Síncrona] Consumo Neto de RAM: 64.75 MB

```

Resultados de la Tarea 3:

```

const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
console.log('[Lectura por Stream] Iniciando procesamiento...');
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
let totalRegistros = 0;
// Creamos un flujo de lectura secuencial no bloqueante
const readableStream = fs.createReadStream(FILE_NAME, { encoding: 'utf-8' });

// Ahorro del Garbage Collector:
// El 'chunk' vive solo durante la ejecución del callback. Al terminar,
// ya no existe ninguna referencia a él, por lo que V8 puede liberarlo en una recolección rápida
// de la generación joven, sin recorrer un heap grande, lo que ahorra el coste de recolección.
// Lo único que persiste entre un trozo y otro es la variable totalRegistros.

readableStream.on('data', (chunk) => {
  // Procesamos el chunk de datos actual sin almacenarlo completo

  let lineBreakCount = (chunk.match(/\n/g) || []).length;
  totalRegistros += lineBreakCount;
});

readableStream.on('end', () => {
  const end = performance.now();
  const memoryAfter = process.memoryUsage().heapUsed;
  console.log('[Lectura por Stream] Total registros procesados: ${totalRegistros-1}');
  console.log('[Lectura por Stream] Tiempo de I/O parcializado: ${((end - start) / 1000).toFixed(2)} segundos.~');
  console.log('[Lectura por Stream] Consumo Neto de RAM: ${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB~');
});

```

```

[Lectura por Stream] Iniciando procesamiento...
[Lectura por Stream] Total registros procesados: 1000000
[Lectura por Stream] Tiempo de I/O parcializado: 0.06 segundos.
[Lectura por Stream] Consumo Neto de RAM: 0.75 MB

```

Comparación de los resultados de la tarea 2 y 3:

Tarea	Tiempo de ejecución	Consumo de RAM
lab02_lectura_sync.js	0.10 s	64.75 MB
lab02_lectura_stream.js	0.07 s	0.66 MB

La razón del consumo de RAM tan diferente es debido a la manera en la que se almacenan y leen los datos. Mientras que, con una lectura síncrona, en el programa se tienen que cargar todos los datos a la vez para después poder procesarlos, además de que estos se dividen en Strings, en una lectura de stream todo va por partes, se guarda un trozo en la memoria y se procesan los datos, para después pasar al siguiente y eliminar el anterior. En una situación el consumo es proporcional al tamaño del archivo $S(n) = O(n)$, en otra, al trabajar en trozos, su consumo se mantiene constantes $S(n) = O(1)$.

Respuestas a la Tarea 4:

Gestión de llamadas

Node.js ejecuta el JavaScript en un solo hilo principal. Con el código de la Tarea 2, la lectura se hace en el propio hilo principal, que queda bloqueado hasta que el archivo completo llega a la RAM, lo que produce el bloqueo. En cambio, en el código de la Tarea 3, gracias a libuv y sus hilos de trabajo, la petición de lectura se va al Thread Pool y un hilo de trabajo solicita al sistema operativo cada trozo, los datos viajan del disco a la RAM por el bus de datos y, al completarse cada lectura, se ejecuta el callback presente en el código. Así, el hilo principal queda libre para atender otras tareas.

Razón de la diferencia en el consumo de memoria

Basándonos en los códigos de las tareas, con `readFileSync` se carga el archivo completo en memoria y con `split("\n")` se crea un string por cada línea.

Con b , como la cantidad de bytes que ocupa un registro en el archivo, n , como el número de registros, y o , como el overhead por línea.

$$S(n) = b \cdot n + o \cdot n = (b + o) \cdot n$$

El primer término es el texto completo y el segundo es el overhead de las líneas. Tomando $c = b + o$ y $n_0 = 1$, $S(n)$, que representa la memoria, es $\leq c \cdot n$ para todo $n \geq n_0$. Por lo tanto, $S(n)$ es igual a $O(n)$, que representa un crecimiento lineal, y nos indica que la memoria crece a medida que lo hace la entrada de datos (n).

Mientras que con `Stream` en cada instante sólo existe en memoria un trozo de tamaño fijo B , más unas pocas variables de k bytes, que utiliza el programa.

$$S(n) = B + k$$

Esta expresión no depende de n , por lo que $S(n) \leq (B + k) \cdot 1$ para todo $n \geq 1$. $S(n)$ al ser igual que $O(1)$, que representa que no hay crecimiento, nos muestra que, al no involucrarse n , la memoria no crece y se mantiene constante.

Con $O(n)$, el heap se llena de objetos que el recolector de basura debe recorrer y liberar, lo que consume CPU adicional, por lo que ocasiona más consumo eléctrico y más calor.

Con $O(1)$, el heap se mantiene pequeño, el recolector trabaja menos y hay menos calor, pues el consumo de memoria no crece con el tamaño del archivo.

Al procesar muchos archivos, esa diferencia se acumula y reduce el costo energético.

7. Preguntas de Control:

1. **Causa Material:** Al realizar una lectura síncrona mediante `readFileSync`, ¿cuál es la "materia" que se satura en el ordenador y cómo se diferencia físicamente del estado de reposo de un archivo almacenado magnética o electrónicamente en el disco duro?

La materia que se puede saturar es la memoria RAM, porque al utilizar `readFileSync()` se carga el contenido completo del archivo en la memoria para que el programa pueda trabajar con los datos.

La diferencia radica en que los discos duros (HDD) y las unidades de estado sólido (SSD) almacenan la información de forma permanente, incluso cuando se apaga el computador. En cambio, la memoria RAM almacena temporalmente los datos que se utilizan durante la ejecución de un programa. Por eso se considera volátil, ya que pierde su contenido cuando se interrumpe el suministro eléctrico.

2. **Causa Formal:** El archivo CSV (Valores Separados por Comas) posee una "forma" específica (texto secuencial lineal). ¿De qué manera esta forma impone restricciones algorítmicas sobre nuestra capacidad para acceder aleatoriamente a la coordenada

$N = 500,000$ sin tener que recorrer las ~~499,999~~ coordenadas previas?

La forma secuencial del archivo CSV impone una restricción porque los datos se almacenan uno detrás de otro, en un orden lineal. Para encontrar una coordenada específica, normalmente se debe recorrer el archivo desde el inicio hasta llegar al dato deseado, ya que no se puede acceder directamente a una fila sin conocer su posición exacta en el archivo.

Si tenemos $N = 500\,000$ coordenadas, buscar la última puede requerir recorrer las 499 999 coordenadas anteriores. Esto genera un tiempo de búsqueda de complejidad $O(N)$.

Para solucionar este problema, se puede utilizar una estructura de datos indexada que permita acceder directamente a la posición de cada coordenada, sin necesidad de recorrer todos los registros anteriores.

3. Causa Eficiente: Durante el uso de *Streams* asíncronos en Node.js, el *Event Loop* delega la operación al sistema operativo. En este contexto, ¿quién actúa como el verdadero motor (causa eficiente) que transfiere los electrones desde el disco hasta la memoria RAM en fragmentos (chunks)?

Dentro de este contexto el verdadero motor físico que impulsa y transfiere físicamente los fragmentos de datos o “chunks” desde el disco hacia la memoria RAM es el “Controlador de Acceso Directo a Memoria” (DMA Controller) que opera en coordinación con el controlador del disco.

Cuando un stream asíncrono en Node.js solicita leer algún archivo, el Event loop le da la operación a la biblioteca libuv, quien se encarga de gestionar la llamada al sistema para que el sistema operativo y el controlador del dispositivo preparen el hardware. Al instante el controlador DMA toma el control del bus del sistema y transfiere directamente los chunks a la memoria RAM sin cargar a la CPU con el movimiento de cada byte. Con todo el proceso anterior, cuando el chunk es depositado en la memoria RAM, el controlador DMA emite una interrupción de hardware a la CPU, lo que permite que el sistema operativo le notifique a libuv y el Event Loop reanude la ejecución del callback en JavaScript con todos los datos ya disponibles.

4. Causa Final: Si todo dato en ejecución debe residir en la memoria principal para ser computado por la CPU, ¿cuál es el propósito (telos) de la "persistencia" en el disco físico, sabiendo que las operaciones de I/O añaden un costo de tiempo monumental en la escala de microsegundos (μs) frente a los nanosegundos (ns) de la memoria caché?

La memoria RAM (DRAM) es un medio volátil que requiere energía eléctrica constante para mantener la información en sus capacitores y transistores. Si la computadora se apaga, se reinicia o se corta la luz, los datos en la RAM desaparecen por completo. El disco físico ya sea las celdas flash de un SSD o los platos de un HDD ofrece un medio no volátil. Su propósito final es garantizar la supervivencia de los datos permitiendo que el sistema pueda apagarse o recuperarse de un fallo y

volver a cargar el estado exacto de la información sin perder nada. Teniendo en cuenta la solución podemos plantear otro inconveniente, los costos, existe una limitación física y económica, la memoria RAM es rápida y orden de nanosegundos, **ns**, pero es muy costosa y de capacidad limitada. El disco físico es más lento ordena en microsegundos, **μs**, pero nos ofrece almacenamiento masivo a un costo bajo. Podemos decir que el disco actúa como el depósito principal de datos para vencer la falta de espacio en la RAM. Gracias al principio de localidad, el sistema operativo solo trae a la memoria los datos que la CPU necesita en el momento. Aunque acceder al disco toma microsegundos (**μs**) y la RAM responde en nanosegundos (**ns**), pagar ese costo de latencia es necesario para procesar archivos gigantescos. Al utilizar Streams en Node.js, leemos el archivo en pequeñas partes directamente desde el disco. Esto evita saturar la RAM, ahorra memoria y reduce el trabajo del Garbage Collector, disminuyendo el consumo energético del procesador.

5. Esencia y Existencia: Un conjunto de 10^6 coordenadas existe como un bloque monolítico en el archivo data.csv, pero en el enfoque de Streams, solo una porción existe en la RAM a la vez. ¿Es conceptualmente el "mismo" conjunto de datos procesado, si en la memoria jamás llegó a existir de manera unificada? Argumente.

Sí llega a ser el mismo conjunto de datos, aunque sólo lo haga en trozos, su contenido pasa de manera consecutiva por la RAM, los mismos datos, con su orden, valor y elementos son los mismos y no cambian. Y su existencia, pese a que en la RAM no está completa, sí existen los datos dentro del disco duro, por lo que la información procesada es la misma aunque no se lean todos los datos a la vez. Sí necesitamos que la información esté en trozos o se lea toda la información a la vez, la cosa cambia dependiendo de la operación a realizar con los datos.

8. Conclusiones

La manera en la que se almacenan los datos en la memoria RAM varía dependiendo de la forma en la que se le permita a un programa acceder a ellos, con `readFileSync` todo el archivo se carga en memoria y el espacio crece con el tamaño de la entrada $O(n)$, y con `Stream`, solo hay un trozo a la vez y el espacio se mantiene constante, $O(1)$. De la misma forma, dependiendo de ello, el gasto energético y el trabajo del garbage collector pueden variar, pues el trabajo de este último depende del tamaño del heap. Por eso, quien trabaje con datos debe pensar en la manera adecuada de procesarlos, pues tienen propósitos y usos en contextos distintos.

Además, la forma de procesar los datos determina qué ocurre con el programa durante la lectura. Con `readFileSync`, el hilo principal queda bloqueado hasta que el archivo completo llega a la RAM. Con `Stream`, la lectura se delega al Thread Pool de libuv y al sistema operativo, que se comunica con el disco duro, mientras el hilo principal sigue libre y se ejecuta un callback cada vez que llega un trozo.

Tener en cuenta estos conceptos ayuda a saber cómo trabajar con datos y a reconocer fallos y mejoras que se puedan hacer a un programa.

9. Bibliografía

- P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023. [Online]. Available: <https://open.palni.org/>
- Node.js Foundation, "File system | Node.js v20.x Documentation", *Node.js API*, 2023. [Online]. Available: <https://nodejs.org/docs/latest-v20.x/api/fs.html>
- S. Tilkov et al., *Node.js in Action*, 2nd ed. Manning Publications, 2017.
- D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5.^a ed. Waltham, MA, EE. UU.: Morgan Kaufmann, 2014.
- Node.js, «The Node.js Event Loop, Timers, and process.nextTick()», *Node.js Learn*. [Online]. Available: <https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick>
- A. Silberschatz, P. B. Galvin, y G. Gagne, *Operating System Concepts*, 10.^a ed. Hoboken, NJ, EE. UU.: Wiley, 2018.
- libuv, «Design overview», *libuv Documentation*. [Online]. Available: <https://docs.libuv.org/en/v1.x/design.html>
- L. Joyanes Aguilar, *Fundamentos de programación: Algoritmos, estructura de datos y objetos*, 4.^a ed. Madrid, España: McGraw-Hill, 2008.
- libuv Project, "Thread pool work scheduling," *libuv documentation*. [En línea]. Disponible: <https://docs.libuv.org/en/latest/threadpool.html>
- OpenJS Foundation, "Overview of blocking vs non-blocking," *Node.js Learn*. [En línea]. Disponible: <https://nodejs.org/learn/asynchronous-work/overview-of-blocking-vs-non-blocking>
- P. Marshall, "Trash talk: the Orinoco garbage collector," *V8 Blog*, enero. 2019. [En línea]. Disponible: <https://v8.dev/blog/trash-talk>

10. Anexos

Enlace a GitHub: <https://github.com/B3kk4r/Estructura-de-datos-Grupo9.git>

Enlace al CSV generado por el código:

https://drive.google.com/drive/folders/1y2m7eQABVVWYASefG9zR_PCOOC3gm-DE?usp=sharing